React aplicado a TimeShift: arquitectura de componentes, flujo de datos y decisiones técnicas

Boris Armando Campos López, Diego Humberto Márquez Valencia, Marcos Abdulio López Alvarado y Alan Enrique Ventura Hernández

Facultad de Ingeniería, Universidad Don Bosco

DAW901: Desarrollo de Aplicaciones Web con Software Interpretado en el Cliente

Alexander Sigüenza

8 de noviembre de 2026

React aplicado a TimeShift: arquitectura de componentes, flujo de datos y decisiones técnicas

Contexto del proyecto

TimeShift es un sistema de control de asistencia y de estados auxiliares (AUX) desarrollado para la Oficina Jurídica Jovita Alvarado. Cada empleado registra en qué estado se encuentra durante la jornada y la administración observa esa información en tiempo real. A solicitud de la cliente, el sistema maneja cuatro estados: Disponible, Lunch, Break y Salida.

La primera versión se construyó con HTML, CSS y JavaScript sin framework. Esa versión cumplió su propósito, pero el equipo detectó problemas de mantenimiento: la validación de sesión (la función *tsLogout*) estaba copiada en cuatro páginas (*dashboard.html*, *gestion de datos.html*, *portal.html* y *reportes.html*) y las pantallas no compartían un único sistema visual. Para este foro, el equipo migró el sistema a React con el fin de reutilizar componentes, centralizar el acceso a datos y proteger las rutas por rol en un solo lugar.

Este documento describe la arquitectura resultante, el flujo de datos entre sus módulos, las decisiones técnicas tomadas y el estado real de la migración. El código fuente se encuentra en el repositorio del proyecto (Equipo TimeShift, s. f.).

Marco tecnológico

La Tabla 1 resume las herramientas empleadas. Las versiones corresponden al archivo *package.json* de la aplicación.

Tabla 1
Tecnologías de la aplicación React

Herramienta	Versión	Función en TimeShift
React	19	Construcción de la interfaz con componentes y estado
React Router	7	Rutas, rutas anidadas y redirecciones por rol
Vite	8	Servidor de desarrollo y empaquetado
Tailwind CSS	4	Estilos mediante utilidades y variables de tema
ESLint	10	Análisis estático del código

Nota. Fuente: elaboración propia a partir de package.json del proyecto.

React permite describir la interfaz como funciones que devuelven elementos y conservar información entre renderizados mediante el estado de cada componente (Meta Platforms, s. f.-a). Para sincronizar un componente con algo externo a React, como un temporizador, se utiliza el efecto *useEffect*, que además admite una función de limpieza (Meta Platforms, s. f.-b). React Router ofrece el enrutamiento declarativo del lado del cliente (React Router, s. f.). Vite se eligió por su servidor de desarrollo rápido y su empaquetado para producción (Vite, s. f.), y Tailwind CSS por permitir definir variables de tema que generan utilidades de color y tipografía (Tailwind Labs, s. f.).

Como el proyecto no cuenta con servidor propio, los datos se guardan en el navegador mediante *localStorage*, una de las interfaces de la API Web Storage que almacena pares clave-valor que persisten entre sesiones (MDN Web Docs, s. f.).

Arquitectura de la aplicación

Organización del código

El código se divide por responsabilidad, de modo que la interfaz no accede directamente al almacenamiento (Tabla 2).

Tabla 2
Carpetas principales de src

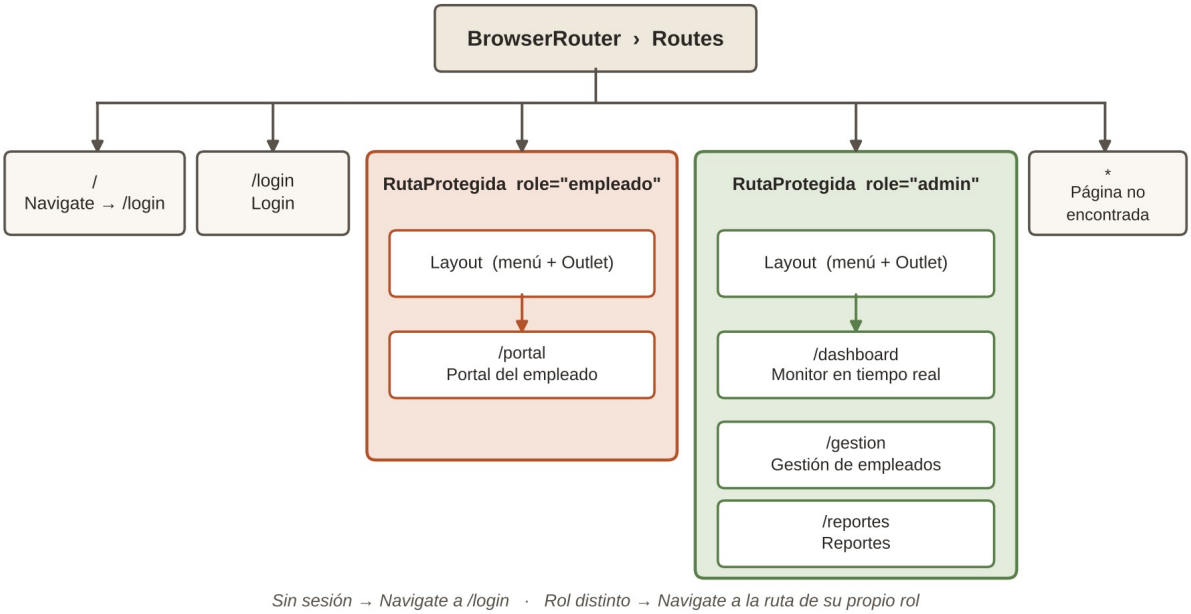
Carpeta	Contenido
`components/`	Piezas reutilizables: Layout, RutaProtegida, Card, StateBadge
`pages/`	Pantallas: Login, Portal, Dashboard (Monitor), Gestión y Reportes
`services/`	Acceso a datos: auth.js (usuarios y sesión) y status.js (estados e historial)
`constants/`	states.js: definición de los cuatro estados y su color
`utils/`	time.js: formato de duración HH:MM:SS

Nota. Fuente: elaboración propia a partir del repositorio.

Rutas y control de acceso

La Figura 1 muestra el árbol de rutas definido en *App.jsx*. Las rutas del administrador y del empleado están agrupadas en dos componentes *RutaProtegida*, que envuelven al *Layout* común. Una ruta desconocida muestra el mensaje “Página no encontrada”.

Figura 1
Árbol de rutas y protección por rol



Nota. Elaboración propia a partir de App.jsx. Las pantallas /gestion y /reportes, la integración de la API externa y los estilos finales se unieron a la rama principal (main) mediante el pull request n.º 13.

El componente *RutaProtegida* aplica dos reglas: si no hay sesión, redirige a */login*; si la hay pero el rol no coincide, redirige a la ruta propia de ese rol. Con ello, la verificación que antes se repetía en cuatro páginas queda en un solo archivo (Figura 2 muestra el resultado esencial del código).

Figura 2
Lógica de RutaProtegida (versión simplificada)

```
const session = getSession();
if (!session) return <Navigate to="/login" replace />;
if (session.role !== role)
  return <Navigate to={routeForRole(session.role)} replace />;
return children;
```

Nota. Código simplificado de components/RutaProtegida.jsx; se omiten importaciones y la declaración de la función.

Migración a React: base, Portal y Monitor

Sección elaborada con base en el trabajo de Boris Armando Campos López (rama boris/portal-monitor). Los datos fueron contrastados con el código del repositorio.

Base del proyecto. La aplicación se creó con Vite y React 19. La navegación usa React Router, con rutas protegidas por rol: el administrador entra al Monitor, Gestión y Reportes, y el empleado entra a su Portal. Para el diseño se instaló Tailwind CSS 4 y se definieron tokens de color y tipografía en *index.css* (azul institucional, un color por estado AUX y las fuentes Inter e IBM Plex Mono). Así, si el cliente pide otro color, se cambia en un solo archivo. Los cuatro estados (Disponible, Lunch, Break y Salida) están en una sola constante, *states.js*, que usan todas las pantallas.

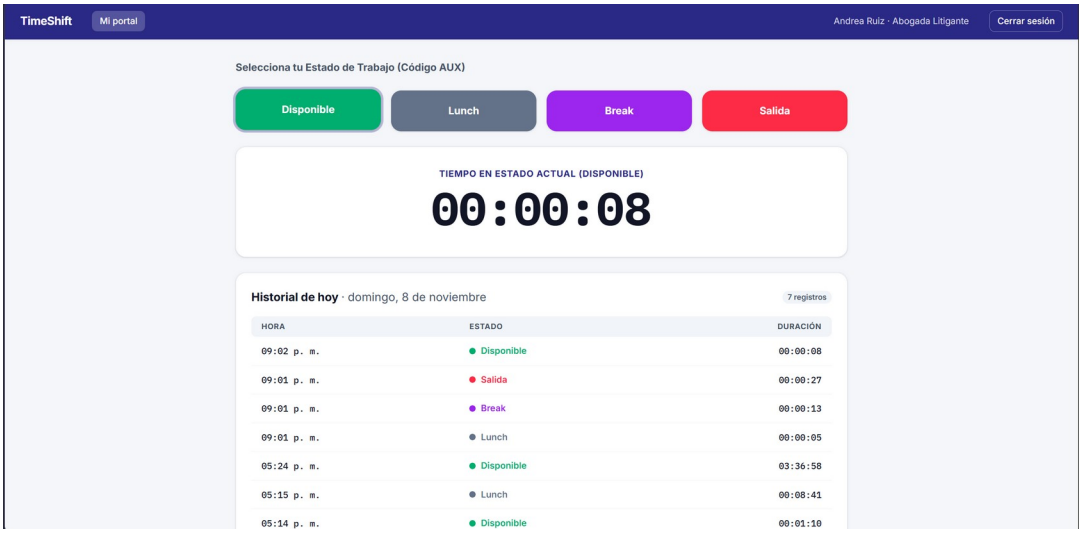
Portal del empleado. El agente elige su estado con cuatro botones y un cronómetro muestra el tiempo en el estado actual. El tiempo no se cuenta sumando segundos: se calcula como la diferencia entre la hora actual y la hora de inicio, así que se mantiene exacto aunque el navegador retrase el intervalo. El estado y el historial del día se guardan en *localStorage* por usuario y por fecha, por lo que sobreviven a recargar la página. El historial muestra la hora, el estado y la duración de cada tramo, y antes de registrar la Salida se pide confirmación.

Monitor en tiempo real. El administrador ve cuántos empleados hay en cada estado y una tabla con cada empleado, su estado y el tiempo transcurrido, con buscador y filtro. El Monitor vuelve a leer *localStorage* cada segundo, por eso un cambio hecho en el Portal aparece sin recargar, incluso desde otra pestaña.

Componentes reutilizables. Se crearon *Card* (contenedor) y *StateBadge* (indicador de color del estado), usados tanto en el Portal como en el Monitor. Un cambio de diseño se aplica en un solo lugar.

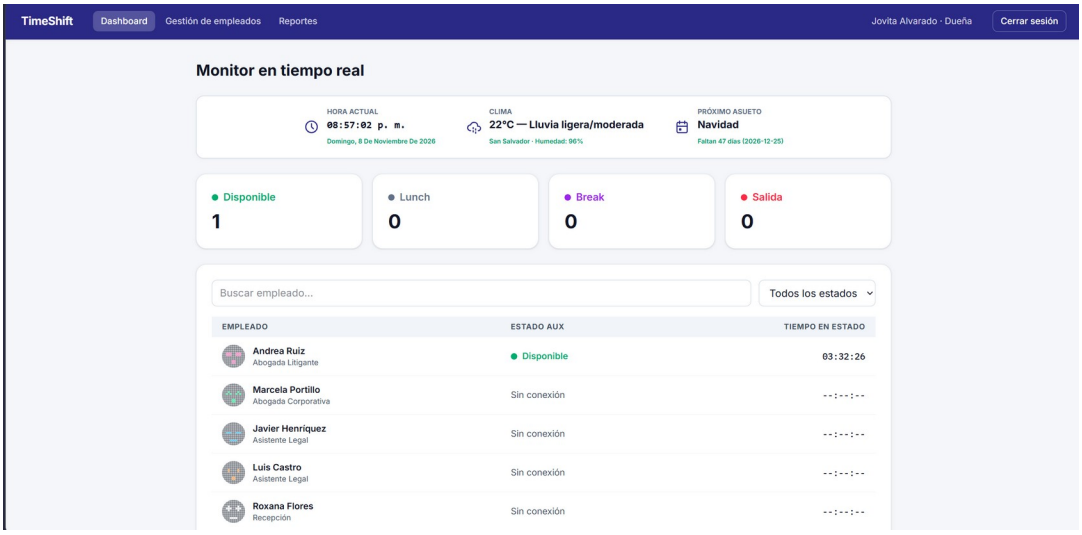
Buenas prácticas aplicadas. Hooks *useState* y *useEffect* con limpieza del intervalo, funciones puras separadas en *utils/* y *services/*, manejo de datos dañados en *localStorage* con *try/catch*, ESLint sin errores y commits descriptivos en una rama propia.

Figura 3
Portal del empleado



Nota. Captura de la aplicación integrada, ejecutada en local con datos de prueba.

Figura 4
Monitor en tiempo real



Nota. Captura de la aplicación integrada, ejecutada en local con conexión a internet: reloj del navegador, clima de Open-Meteo, próximo asueto y avatares de DiceBear.

Figura 5
Portal del empleado en pantalla de teléfono

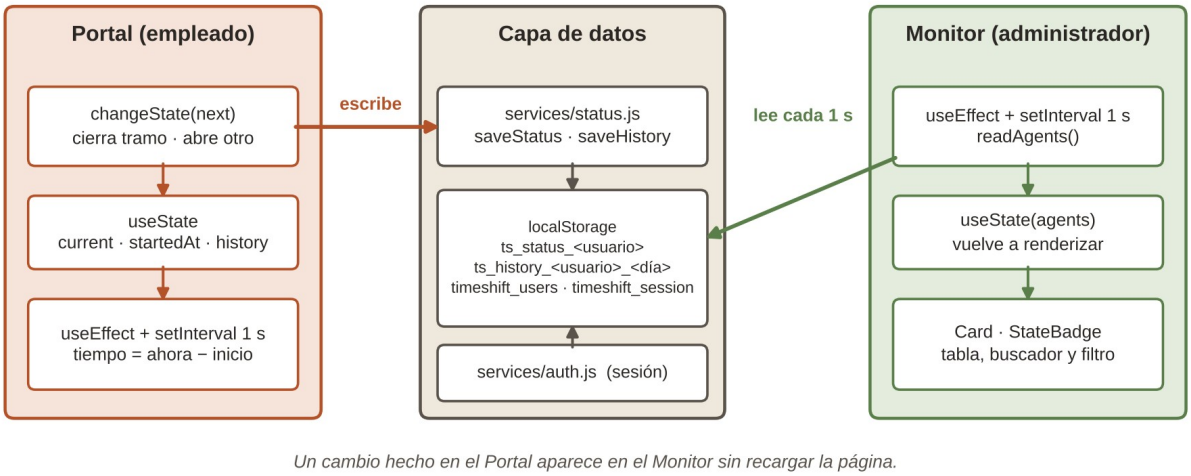


Nota. Captura con datos de prueba en un ancho de 390 píxeles.

Flujo de datos y manejo del estado

La comunicación entre el Portal y el Monitor no ocurre directamente: ambos pasan por la capa de servicios y por *localStorage* (Figura 6). El Portal escribe el estado y el historial; el Monitor los lee cada segundo y vuelve a renderizar.

Figura 6
Flujo de datos entre Portal, capa de datos y Monitor



Nota. Elaboración propia a partir de Portal.jsx, Dashboard.jsx y los servicios.

Los datos se organizan con las claves de la Tabla 3.

Tabla 3
Claves de localStorage

Clave	Contenido
`timeshift_users`	Usuarios del sistema (con valores por defecto)
`timeshift_session`	Usuario con sesión activa y su rol
`ts_status_<usuario>`	Estado actual y hora de inicio de cada empleado
`ts_history_<usuario>_<AAAA-MM-DD>`	Tramos de estado de un empleado en un día

Nota. Fuente: elaboración propia a partir de auth.js y status.js.

Existen dos formas de mantener sincronizadas varias pestañas: escuchar el evento *storage* del navegador o consultar periódicamente. El equipo eligió la consulta cada segundo porque es simple y permite tener el Monitor y el Portal abiertos en pestañas distintas del mismo navegador; el costo es una lectura constante de datos pequeños, aceptable para el volumen de una oficina.

Decisiones técnicas

Tabla 4
Decisiones y motivos

Decisión	Motivo	Alternativa descartada
Migrar a React	Reutilizar componentes y centralizar la sesión	Seguir con HTML por página
Rutas protegidas por rol	Un solo punto de control de acceso	Validar en cada página
Capa de servicios	Aislar el almacenamiento de las pantallas	Leer localStorage en cada componente
Consulta cada 1 s	Simplicidad para el monitor	Evento storage
Cuatro estados AUX	Requisito de la cliente	At Court y Client Meeting del mockup
Tailwind con variables de tema	Un solo sistema visual	CSS distinto por página

Nota. Fuente: elaboración propia. Los motivos fueron redactados a partir del código y deben ser confirmados por el equipo.

Módulos secundarios y estado de la aplicación

Esta sección corresponde al trabajo de Marcos Abdulio López Alvarado (rama feature/modulos-secundarios), que migró a React la gestión de empleados y los reportes. Este código ya forma parte de la rama principal.

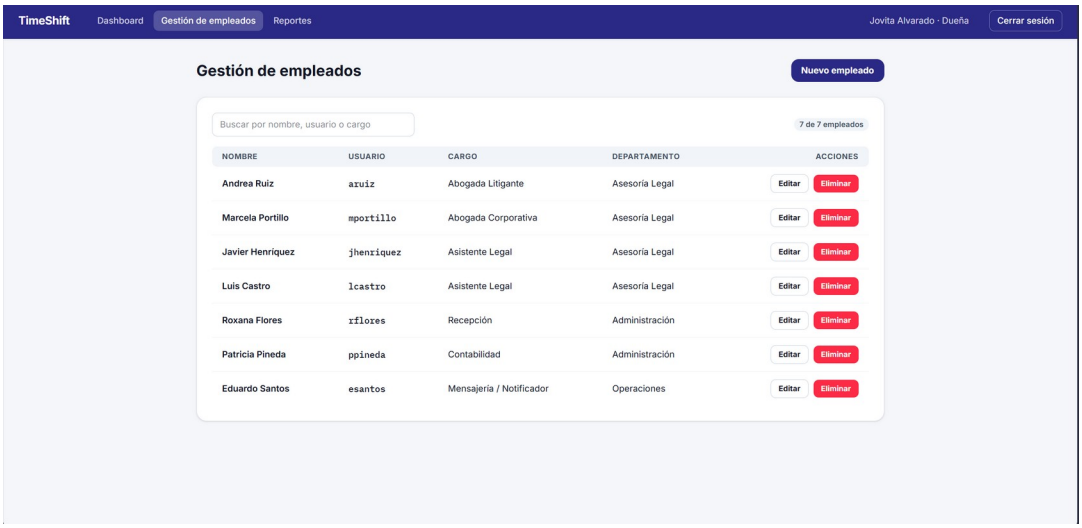
Estado global de empleados. La lista de usuarios se maneja con un reducer (*useReducer*) dentro de un proveedor de contexto, *EmpleadosProvider*, que envuelve el *Layout* del administrador. El reducer admite tres acciones: agregar, actualizar y eliminar. Cada cambio se guarda en *localStorage* con un *useEffect* bajo la misma clave que usa el inicio de sesión (*timeshift_users*), de modo que un empleado nuevo puede iniciar sesión de inmediato. Las pantallas acceden a este estado con el hook *useEmpleados*, que lanza un error claro si se usa fuera del proveedor.

Gestión de empleados. El formulario valida con funciones de *services/empleados.js*: nombre de 3 a 60 caracteres; usuario de 3 a 20 caracteres con minúsculas, números, punto, guion o guion bajo, sin duplicados; cargo obligatorio de hasta 60 caracteres; y contraseña de al menos 8

caracteres con letras y números (al editar, dejarla vacía significa no cambiarla). Al eliminar un empleado también se borra su estado guardado del Portal. La Figura 7 muestra la pantalla.

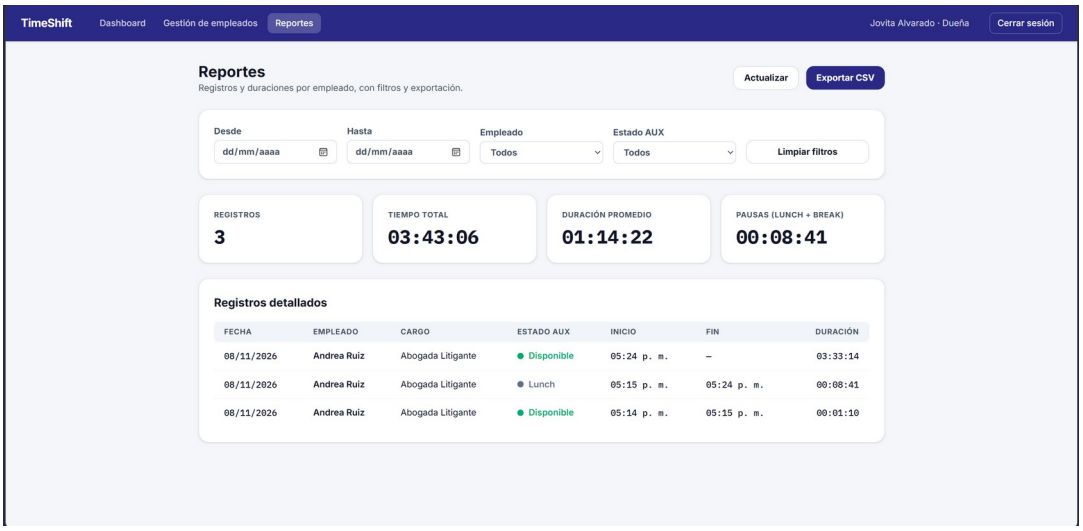
Reportes. *services/reportes.js* reconstruye las filas del reporte leyendo del *localStorage* los historiales que escribe el Portal (claves *ts_history_<usuario>_<día>*). La pantalla permite filtrar por rango de fechas, empleado y estado, muestra un resumen y exporta el resultado a CSV. “Salida” se trata como marca de fin de turno y no lleva duración. La Figura 8 muestra la pantalla.

Figura 7
Gestión de empleados



Nota. Captura de la aplicación integrada con los siete empleados de prueba.

Figura 8
Reportes



Nota. Captura de la aplicación integrada con los registros del empleado de prueba del día.

Gestión de empleados en la versión HTML

Alan Enrique Ventura Hernández desarrolló el módulo de gestión de empleados de la versión HTML. Permite agregar, editar y eliminar con validaciones de nombre, correo, cargo y horario de descanso, y usa SweetAlert2 para los avisos y confirmaciones. Los datos se escapan antes de mostrarse en la tabla para evitar la inserción de HTML. Las reglas de validación de la versión React son las de Marcos, descritas arriba.

Integración con una API externa

Esta sección corresponde al trabajo de Diego Humberto Márquez Valencia (rama feature/Consumo-de-API). Centralizó las peticiones externas en el módulo *services/externalApi.js*, que usa *fetch* con *async/await*, y las muestra en el panel del administrador. La Tabla 5 resume los servicios que consume.

Tabla 5
Servicios externos consumidos

Servicio	Dato	Si falla
Open-Meteo	Temperatura, humedad y condición del clima en San Salvador	Muestra una estimación local
Nager.Date	Próximo asueto nacional	Usa el calendario oficial guardado en el código
RandomUser	Datos de un empleado de prueba para el formulario de Gestión	Usa un nombre de una lista local
DiceBear	Avatar de cada empleado en la tabla del Monitor	Genera un avatar SVG con sus iniciales

Nota. Fuente: elaboración propia a partir de services/externalApi.js y de la documentación de cada servicio (DiceBear, s. f.; Nager.Date, s. f.; Open-Meteo, s. f.; RandomUser, s. f.).

Peticiones en paralelo. El componente *ApiWidgets* pide el clima y el próximo asueto al mismo tiempo con *Promise.all*, en lugar de esperar una respuesta antes de pedir la siguiente, y repite la consulta cada diez minutos. Así la pantalla espera solo lo que tarda el servicio más lento y no hace falta un botón para actualizar. El reloj del panel no depende de internet: se calcula con la hora del navegador y se actualiza cada segundo.

Tiempos límite. Cada petición lleva un *AbortController* que la cancela a los cuatro segundos (cuatro y medio para el clima y cinco para RandomUser). Con esto, un servicio lento no deja la pantalla esperando indefinidamente.

Resiliencia. Cada función atrapa sus errores y devuelve un valor alternativo en lugar de lanzar la excepción, de modo que la aplicación sigue funcionando sin internet. Si el servicio del clima falla, se muestra una estimación; si falla el de feriados, se usa el calendario de asuetos oficiales guardado en el código, que además filtra los feriados de la API para conservar solo los asuetos nacionales. Si una imagen de avatar no carga, el evento *onError* la reemplaza por un SVG que dibuja las iniciales del usuario, por lo que no se ven imágenes rotas.

La rama de Diego partió de la base de React por separado y reescribe archivos que también modificaron otras ramas (*Dashboard.jsx*, *Portal.jsx*, *Layout.jsx* y *GestionDatos.jsx*). Para evitar conflictos, en la integración se tomaron solo *externalApi.js* y *ApiWidgets.jsx*, y se agregó el componente al panel del administrador con dos líneas en *Dashboard.jsx*. Después de la integración, Boris Armando Campos López ajustó el panel: reemplazó la consulta de la hora a servicios externos por el reloj del navegador, quitó el botón de actualización, cambió los íconos por los de la biblioteca Tabler Icons (Tabler, s. f.) y mostró el avatar de DiceBear junto al nombre de cada empleado en la tabla del Monitor (Figuras 4 y 9).

Figura 9
Panel de servicios externos en el Monitor



Nota. Captura de la aplicación integrada con conexión a internet: el reloj usa la hora del navegador, el clima proviene de Open-Meteo y el asueto de Nager.Date.

Estado de la migración y limitaciones

Tabla 6
Estado de cada pantalla en la versión React

Pantalla	Estado
Login	Funcional, con estilos aplicados
Portal del empleado	Funcional
Monitor en tiempo real	Funcional
Gestión de empleados	Funcional, integrada a la rama principal
Reportes	Funcional, integrada a la rama principal
Consumo de API externa	Funcional, integrado a la rama principal; con respaldo local si no hay conexión

Nota. Fuente: revisión del repositorio al 8 de noviembre de 2026.

Entre las limitaciones conocidas están que *localStorage* no comparte datos entre equipos distintos, que la sesión también se guarda ahí y por eso todas las pestañas del mismo navegador comparten el mismo usuario, y que las contraseñas de prueba están en el código, lo cual es

aceptable solo para la fase académica; un sistema real requeriría un servidor con autenticación segura.

Conclusiones

La migración a React redujo la duplicación de la validación de sesión, ordenó el código en componentes, servicios y constantes, y permitió que el Portal y el Monitor compartieran datos sin acoplarse. Las pantallas de gestión y reportes y el consumo de servicios externos se integraron a la rama principal. Como trabajo futuro quedan reemplazar *localStorage* por un servidor con base de datos y autenticación segura, y registrar las pruebas automáticas de cada módulo.

Referencias

DiceBear. (s. f.). *HTTP API: Generate SVG avatars via URL*. https://www.dicebear.com/how-to-use/http-api/

Equipo TimeShift. (s. f.). *TimeShift* [Código fuente]. GitHub. https://github.com/Androx2000/TimeShift

MDN Web Docs. (s. f.). *Web Storage API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Meta Platforms. (s. f.-a). *State: A component's memory*. React. https://react.dev/learn/state-a-components-memory

Meta Platforms. (s. f.-b). *useEffect*. React. https://react.dev/reference/react/useEffect

Nager.Date. (s. f.). *Public holiday API*. https://date.nager.at/API

Open-Meteo. (s. f.). *Weather forecast API documentation*. https://open-meteo.com/en/docs

RandomUser. (s. f.). *Random User Generator: Documentation*. https://randomuser.me/documentation

React Router. (s. f.). *Routing*. https://reactrouter.com/start/declarative/routing

Tabler. (s. f.). *Tabler Icons*. https://tabler.io/icons

Tailwind Labs. (s. f.). *Theme variables*. Tailwind CSS. https://tailwindcss.com/docs/theme

Vite. (s. f.). *Why Vite*. https://vite.dev/guide/why